

게임서버 신입프로그래머 황수지

포트폴리오

PROJECTS

1. 티유월드
2. 기타 프로젝트들

PROJECTS

티유월드

한국공대 캠퍼스를 배경으로 한 MMORPG 게임

개발기간

24.10 ~ 25.07

개발인원

3명 (서버 1, 클라이언트1, 모델링1)

담당업무

서버 제작 담당 (기여도 100%)

- 서버 중심 구조의 게임 로직 구현
- IOCP 기반 세션 병렬 처리
- 데이터베이스 스키마 설계 및 연동

클라이언트 보조 (기여도 20%)

- 서버 프로토콜 대응, 네트워크 모듈/패킷 핸들러 구현
- UI 개선 및 일부 로직 구현

개발도구

C++, MySQL, Unreal5.4

라이브러리

MySQL Connector/C++(X DevAPI), spdlog, rapidjson, magic_enum

Git

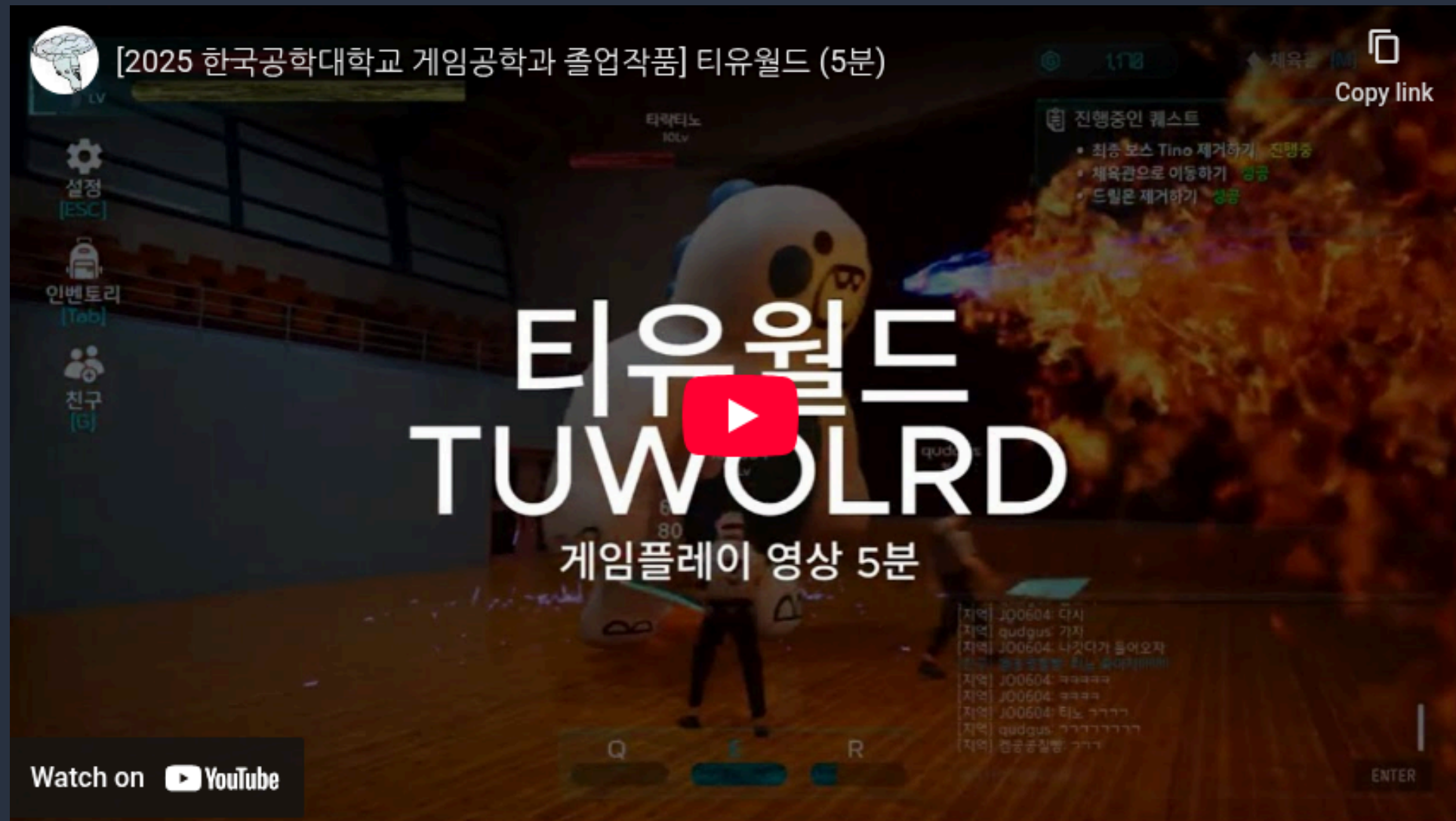
<https://github.com/sujissi/TUWorld.git>



티유월드

티유월드

작품 소개 영상 및 콘텐츠 종류

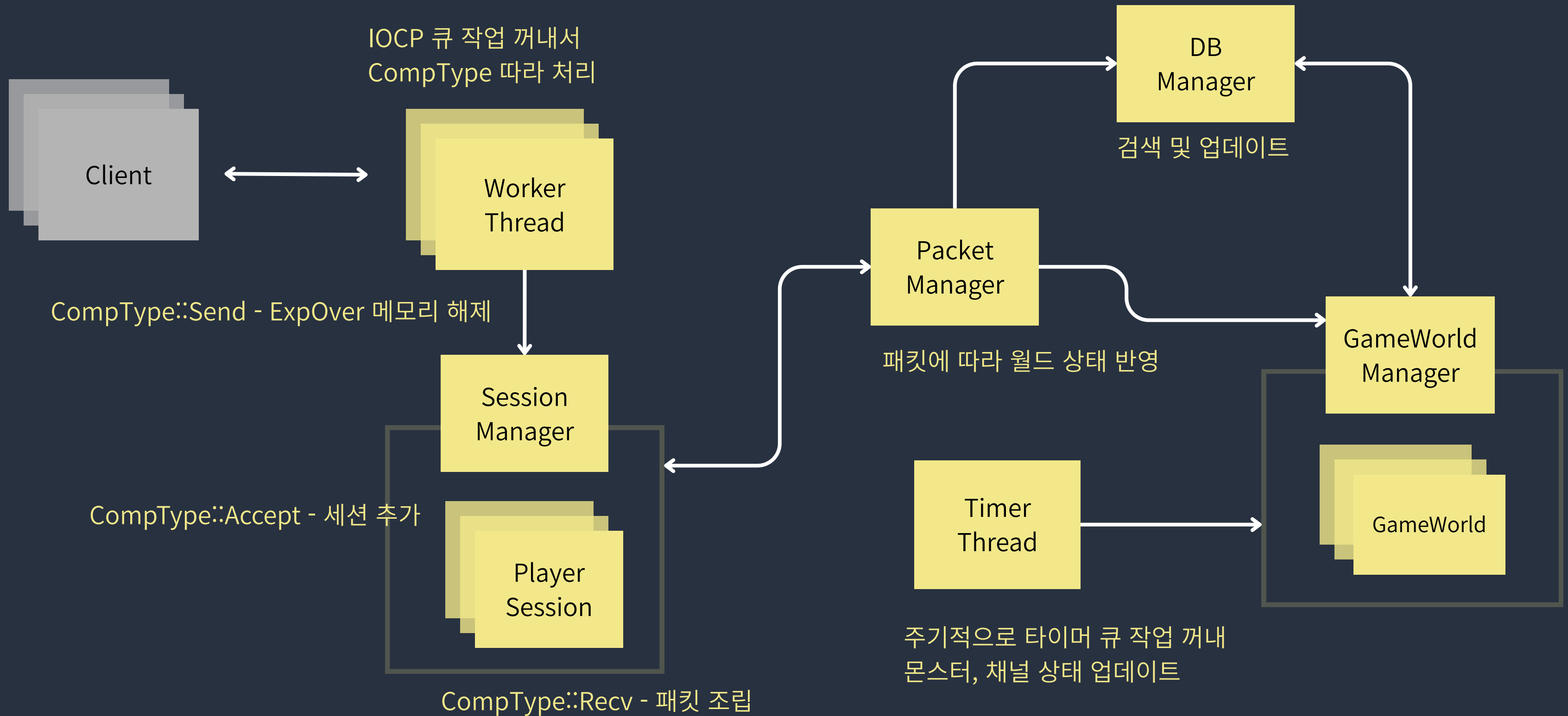


<https://www.youtube.com/watch?v=NkgPZXgjn8c>

한국공대에 나타난 몬스터들을 처치하고
퀘스트를 수행하며 학교의 미스터리를 해결하는 MMORPG

캐릭터	2종
몬스터	10종
스킬	6종 (남 3종, 여 3종)
아이템	23종 (무기/방어구/소모품/퀘스트)
인던	3곳 (학교 건물 4개 중 3곳이 전투 던전, 1곳은 상점)
퀘스트	총 18개 (튜토리얼 퀘스트 4개 + 메인 퀘스트 14개)

티유월드 서버 구조



티유월드

1) 대규모 싼 최적화

어떻게 하면 동접 수를 늘릴 수 있을까?

1. 대규모 접속에 적합한 I/O모델 IOCP를 사용했다.
2. Grid + ViewList로 필요한 범위만 전송했다.
3. 채널링으로 접속 분할, 세션/이벤트를 채널별로 범위 제한했다.

티유월드

1) 대규모 싼 최적화

```
while (_bRunning)
{
    BOOL ret = IOCP::GetInst().GetCompletion(recvByte, sessionId, over);
    ExpOver* expOver = reinterpret_cast<ExpOver*>(over);
    DWORD errorCode = (ret == FALSE) ? GetLastError() : NO_ERROR;

    if (!ret)
    {
        expOver->errorCode = errorCode;
        HandleCompletionError(expOver, static_cast<int32>(sessionId));
        continue;
    }

    switch (expOver->_compType)
    {
        case CompType::ACCEPT:
            SessionManager::GetInst().Connect(_acceptSocket);
            DoAccept();
            break;
        case CompType::RECV:
            if (recvByte == 0)
            {
                SessionManager::GetInst().Disconnect(static_cast<int32>(sessionId));
                break;
            }
            SessionManager::GetInst().OnRecv(static_cast<int32>(sessionId), recvByte, expOver);
            SessionManager::GetInst().DoRecv(static_cast<int32>(sessionId));
            break;
            break;
        case CompType::SEND:
            delete over;
            break;
    }
}
```

비동기적으로 발생하는 **IOCP** 완료 이벤트를
CompType(세션 연결/데이터 수신·송신)따라 병렬 처리
→ 대규모 동시 접속 가능 구조 확보

티유월드 1) 대규모 씬 최적화

```
void GameWorld::PlayerMove(int32 playerId, FVector& newPos, uint32 state, uint64& time)
{
    auto player = GetPlayerByID(playerId);
    if (!player)
    {
        LOG_W("Invaild!");
        return;
    }
    LOG_D("Player [{}] Move {}", playerId, newPos.ToString());
    player->UpdatePos(newPos);
    player->GetInfo().State = static_cast<ECharacterStateType>(state);
    if (player->GetZone() == ZoneType::TUK)
    {
        auto oldPos = player->GetPrevPos();
        UpdateMoveGridInTUK(playerId, oldPos, newPos);
    }
    UpdateViewList(playerId);

    auto pkt = MovePacket(playerId, newPos, state, time, EPacketType::S_PLAYER_MOVE);
    SessionManager::GetInst().SendPacket(playerId, &pkt);
    auto upkt = InfoPacket(EPacketType::S_PLAYER_STATUS_UPDATE, player->GetInfo());
    std::unordered_set<int32> viewList;
    {
        std::lock_guard lock(player->_vlLock);
        viewList = player->GetViewList();
    }
    SessionManager::GetInst().BroadcastToViewList(&upkt, viewList);
}
```

PlayerMove시 TUK(건물 밖 캠퍼스) 지역에서는
Grid 기반 ViewList로 인접 객체만 전송

```
0] [Log] [DummyClient::Connect] Connect success: [D00346] sock=169
01] [Log] [DummyClient::Connect] Connect success: [D00347] sock=169
01] [Log] [DummyClientManager::HandleCompletionError] Skip error ha
```

```
01] [info] [DummyClient::Connect] Connect success: [D01337]
02] [info] [DummyClient::Connect] Connect success: [D01338]
03] [warning] [DummyClientManager::HandleCompletionError] Skip
```

동접 약 4배 증가

티유월드

1) 대규모 씬 최적화

```
void PacketManager::HandleChangeChannelPacket(int32 sessionId, Packet* packet)
{
    auto* p = static_cast<ChangeChannelRequestPacket*>(packet);
    auto session = _sessionMgr.GetSession(sessionId);
    if (!session || !session->IsInGame()) return;

    void PacketManager::HandleEnterGamePacket(int32 sessionId, Packet* packet)
    {
        auto session = _sessionMgr.GetSession(sessionId);
        if (!session || !session->IsLogin()) return;

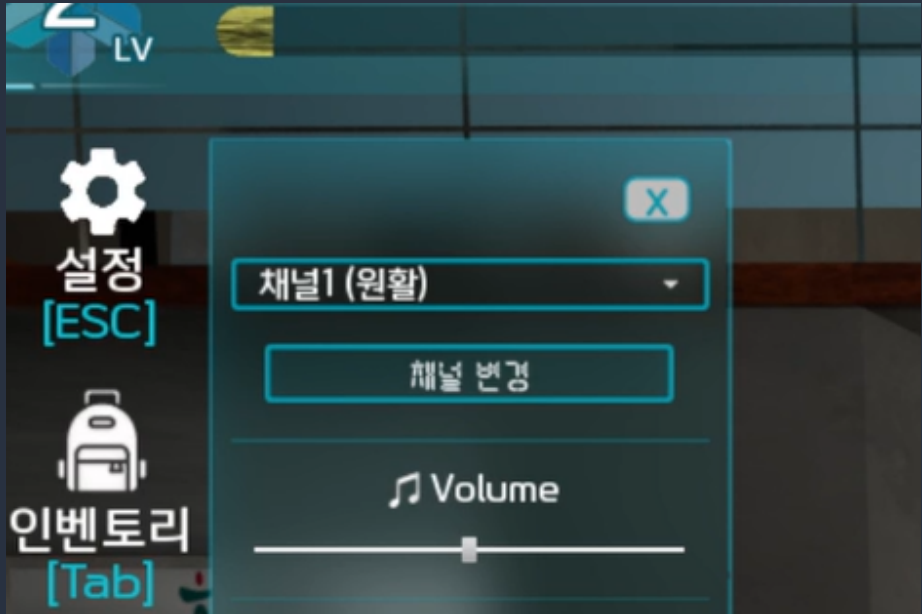
        auto player = session->GetPlayer();
        if (!player) return;

        RequestEnterGamePacket* p = static_cast<RequestEnterGamePacket*>(packet);
        auto newType = p->PlayerType;
        auto wChannelId = p->WChannel;
        GameWorld* world = nullptr;
        if(wChannelId == EWorldChannel::None)
        {
            world = GameWorldManager::GetInst().GetAvailableWorld(wChannelId);
        }
        else
        {
            world = GameWorldManager::GetInst().GetWorld(wChannelId);
        }

        if (!world)
        {
            LOG_W("Invalid World");
            return;
        }
    }
}
```



입장 시 채널 선택



게임 중 채널 변경

```
myClient::Connect] Connect success: [D01337]
myClient::Connect] Connect success: [D01338]
myClient::Connect] Connect success: [D06139]
myClient::Connect] Connect success: [D06140]
```

동접 약 4.5배 증가

티유월드

2) NavMesh 기반 스폰·길찾기

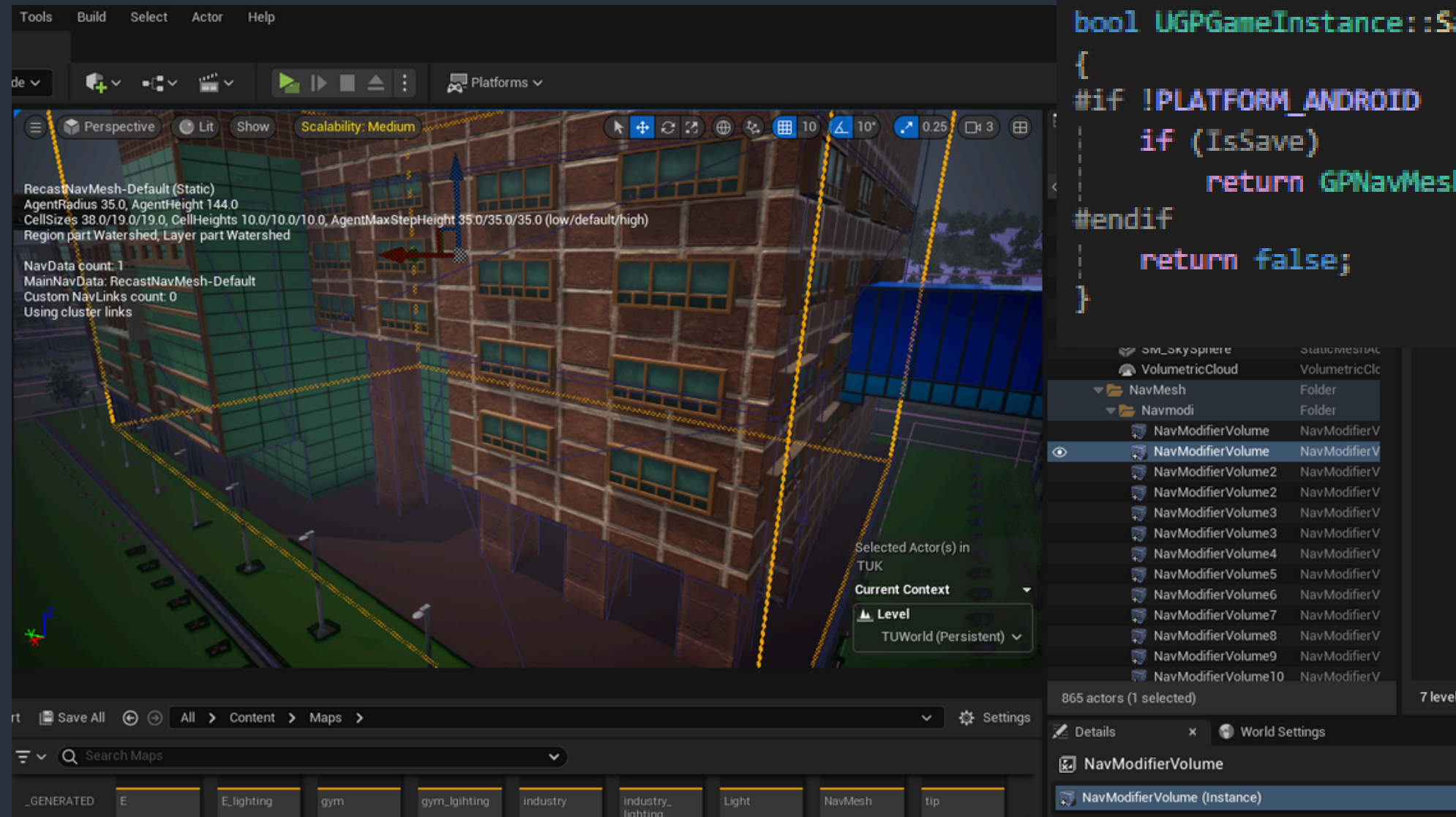
**넓은 범위의 스폰 좌표를 하드코딩 하지 않고,
항상 클라이언트와 일치하게 만들 수는 없을까?**

언리얼 엔진의 내장 NavMesh 데이터를 추출하고

이를 서버에서 로딩해 충돌 판정 및 경로 탐색에 활용하는 시스템을 구현했다.

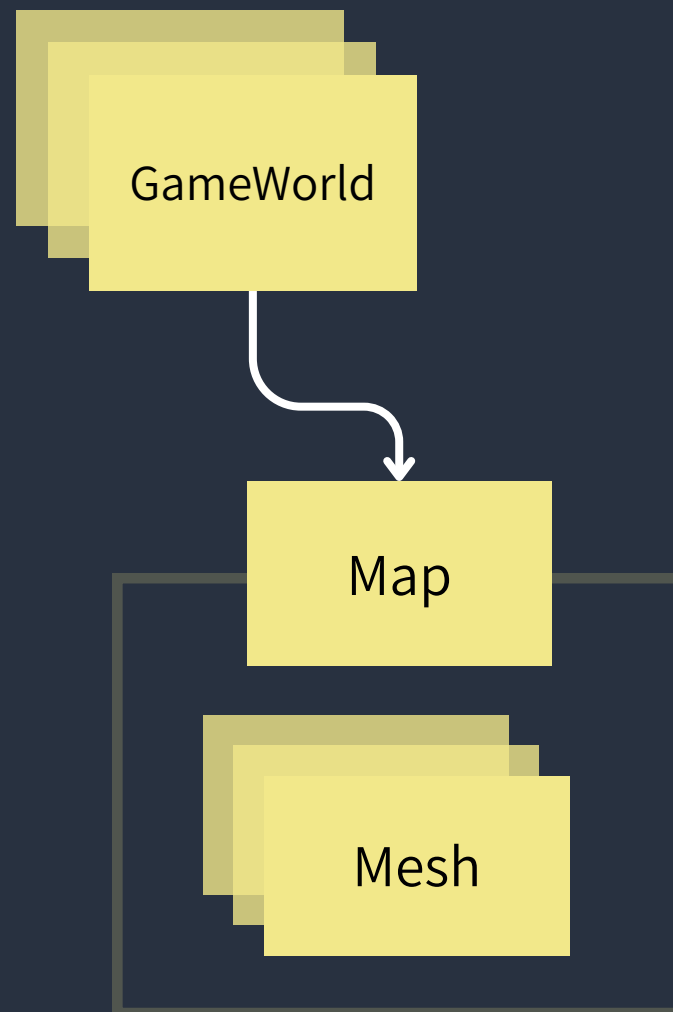
티유월드

2) NavMesh 기반 스폰·길찾기



step1. 언리얼에서 NavMesh볼륨과 Modify볼륨을 배치 후, Gameinstance실행 시 NavMesh(초록영역) 추출

→ 현재 월드의 네비게이션 시스템에서 기본 NavMesh(ARecastNavMesh)를 가져와, 타일을 순회하며 폴리곤·정점을 수집하고 인접 관계를 구성한 뒤 JSON으로 저장



```

class NavMesh
{
public:
    static std::optional<NavMesh> LoadFromJson(const std::string& filePath);
    void Init();
    void BuildPolygonCenters();
    void BuildTriangleAreas();
    void BuildPortalEdges();

    int FindIdxFromPos(const FVector& pos) const;
    int FindClosestPoly(const FVector& pos) const;
    FVector GetRandomPosition() const;
    std::vector<int> FindPathAStar(const FVector& start, const FVector& end) const;
    std::vector<FVector> GetStraightPath(const FVector& start, const FVector& end) const;
    FVector GetNearbyRandomPosition(const FVector& pos) const;

public:
    std::vector<FVector> vertices;
    std::vector<std::vector<int>> poly;
    std::vector<std::vector<int>> neig;
    std::vector<FVector> polyCenters;
    std::vector<TriangleInfo> triangle;
    float triangleTotalArea = 0.f;
    std::unordered_map<std::pair<int, int>, bool> portalEdges;
};

class Map
{
public:
    static Map& GetInst()
    {
        static Map inst;
        return inst;
    }

    bool Init();
    bool IsZoneAccessible(ZoneType zone, uint32 playerLevel) const;

    FVector GetRandomPos(ZoneType type) const;
    NavMesh& GetNavMesh(ZoneType type);

    FVector GetRandomEntryPos(ZoneType oldZone, ZoneType targetZone) const;
    FVector GetRandomEntryPos(EntryType entryType) const;
    NavMesh& GetEntryNavMesh(EntryType entryType);

    FVector GetStartPos(ZoneType startZone);
private:
    std::unordered_map<ZoneType, NavMesh> _navMeshes;
    std::unordered_map<EntryType, NavMesh> _entryMeshes;
  
```

step2. 서버 초기화 시 JSON파일 파싱 및 서버용 NavMesh 빌드 후 Map 클래스에서 활용

→ 플레이어/몬스터 스폰 위치 설정 시, 몬스터 플레이어 쫓을 시 Map 함수 호출

티유월드

2) NavMesh 기반 스폰·길찾기

```
void GameWorld::RespawnPlayer(int32 playerId, ZoneType newZone)
{
    auto player = GetPlayerByID(playerId);
    if (!player)
    {
        LOG_W("Invalid player");
        return;
    }
    ZoneType oldZone = player->GetZone();
    if (oldZone == ZoneType::TUK)
        LeaveGrid(playerId, player->GetPos());
    FVector newPos = Map::GetInst().GetRandomPos(newZone);
    newPos.Z += 90.f;
    auto& info = player->GetInfo();
    info.SetZone(newZone);
    player->UpdatePos(newPos);
    if (newZone == ZoneType::TUK)
    {
        std::lock_guard lock(_gridMutex);
        EnterGrid(playerId, newPos);
    }
    else
```

NavMesh 활용 코드 예 ①

월드 입장/리스폰/맵 전환 시

GetPos로 NavMesh 상 좌표를 얻어옴

```
void GameWorld::PlayerEnterGame(std::shared_ptr<Player> player, bool bChange)
{
    FVector newPos;
    ZoneType startZone = START_ZONE;
    newPos = Map::GetInst().GetStartPos(startZone);
    newPos.Z += 90.f;
    player->UpdatePos(newPos);
```

```
bool GameWorld::TransferToZone(int32 playerId, ZoneType newZone)
{
    LOG_D("Try Transfer");
    auto player = GetPlayerByID(playerId);
    if (!player) return false;
    auto& info = player->GetInfo();
    uint32 playerLevel = info.GetLevel();
    if (newZone == ZoneType::E && !player->HasKey())
    {
        return false;
    }
    if (!Map::GetInst().IsZoneAccessible(newZone, playerLevel))
    {
        LOG_D("Player [{}] cannot access due to level {}", playerId, playerLevel);
        return false;
    }
    ZoneType oldZone = player->GetZone();
    if (oldZone == ZoneType::TUK)
        LeaveGrid(playerId, player->GetPos());

    FVector newPos = Map::GetInst().GetRandomEntryPos(oldZone, newZone);
    if (newPos == FVector::ZeroVector)
    {
        LOG_W("EntryPos");
        return false;
    }
    newPos.Z += 90.f;
```



```
void Monster::Chase()
{
    if (!_target || !_navMesh) return;
    ChangeState(ECharacterStateType::STATE_WALK);
    _info.AddState(ECharacterStateType::STATE_CHASE);

    if (!IsTargetInChaseRange())
    {
        _target.reset();
        ChangeState(ECharacterStateType::STATE_IDLE);
        return;
    }

    FVector start = GetInfo().Pos;
    FVector targetPos = _target->GetInfo().Pos;
    FVector dirToTarget = (targetPos - start).Normalize();
    float safeDist = _info.CollisionRadius + _target->GetInfo().CollisionRadius;
    FVector goal = targetPos - dirToTarget * safeDist;
    goal.Z = targetPos.Z - 90.f;

    auto polyPath = _navMesh->FindPathAStar(start, goal);
    auto pathPoints = _navMesh->GetStraightPath(start, goal, polyPath);
    if (pathPoints.empty())
    {
        ChangeState(ECharacterStateType::STATE_IDLE);
        return;
    }

    _movePath = std::move(pathPoints);
    _pathIdx = 1;
    auto PlayerId = _target->GetInfo().ID;

#ifdef _DEBUG
    for (size_t i = 1; i < _movePath.size(); ++i)
    {
```

몬스터가 해당하는 지역에 따라서 설정된 _navMesh

```
_navMesh = &Map::GetInst().GetNavMesh(_zone);
```

NavMesh 활용 코드 예②

타깃 플레이어를 목표로 설정해 NavMesh 상에서 폴리곤 경로를 구하고,
이를 직선화한 좌표 경로로 변환해 이동에 활용

티유월드

3) 스프레드시트 기반 콘텐츠 관리

**컨텐츠 추가나 게임 밸런스를
팀원도 쉽게 수정할 수 있게 만들 수는 없을까?**

언리얼의 DataTable처럼, 서버에서도 CSV 기반 스프레드시트로 밸런스·컨텐츠를 관리하고,
이를 로드해 유효성 검사와 인덱싱을 수행했다.

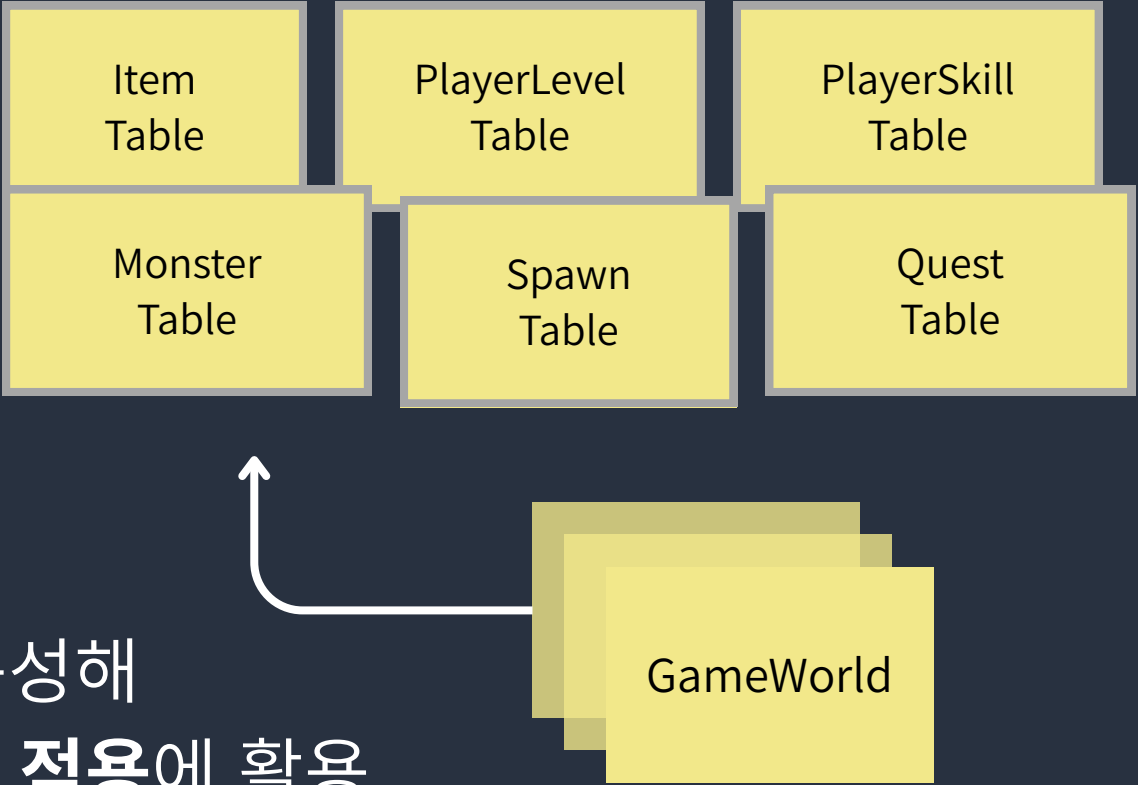
티유월드

3) 스프레드시트 기반 콘텐츠 관리

```
bool res =
    ItemTable::GetInst().LoadFromCSV(DataTablePath + "ItemTable.csv") &&
    PlayerLevelTable::GetInst().LoadFromCSV(DataTablePath + "PlayerLevelTable.csv") &&
    PlayerSkillTable::GetInst().LoadFromCSV(DataTablePath + "PlayerSkillTable.csv") &&
    MonsterTable::GetInst().LoadFromCSV(DataTablePath + "MonsterTable.csv") &&
    SpawnTable::GetInst().LoadFromCSV(DataTablePath + "SpawnTable.csv") &&
    QuestTable::GetInst().LoadFromCSV(DataTablePath + "QuestTable.csv");

if (!res)
{
    LOG_E("LoadFromCSV");
    return false;
}
```

플레이어/몬스터/아이템/퀘스트를 CSV 스키마로 구성해
서버 실행 시 테이블 일괄 로드 해 스폰/드롭/밸런스 적용에 활용



Name	Category	Damage	Hp	CrtRate	Dodge	Speed	AbilityType	AbilityVal	DropRate	FromMon	ResellPrice	Price	isSellable	isBuyable
1 BirdGun	weapon		5	0	0	0	0	0	0.05	1	100	200	1	1
2 PulseGun	weapon	10												
3 Positron	weapon	18												
4 PracSword	weapon	10												
5 PulseSword	weapon	25												
6 EnergySword	weapon	40												
10 AlloyHelm	armor	0												
11 EnhanceH	armor	0												
12 TitanChest	armor	0												
13 EnhanceC	armor	0												
14 Suit	armor	0												
15 TUClothes	armor	0												
20 HpKit_low	useable	0												
21 HpKit_mid	useable	0												
22 HpKit_high	useable	0												
23 Coffee	useable	0												
24 Ddalba	useable	0												
25 Mangba	useable	0												
26 Gold_low	useable	0												
27 Gold_mid	useable	0												
28 Gold_high	useable	0												
50 Key	unuseable	0												
51 Document	unuseable	0												
				QuestID	Chapter	QuestName	Description	Category	ExpReward	GoldReward	RewardItemTypeID	NextQuestID		
				1	1	CH1_TALK_TO_STUDENT_A	TIP에서 학생 A를 만난다	INTERACT	10	5	-1	2		
				2	1	CH1_GO_TO_E_FIRST	E동으로 이동한다	MOVE	10	5	-1	3		
				3	1	CH1_FIND_JANITOR	문이 잠겨있어 경비아저씨를 찾는다	INTERACT	10	5	-1	4		
				4	1	CH1_GO_TO_BUNKER	종합관 앞 병커로 간다	MOVE	10	5	-1	5		
				5	1	CH1_BUNKER_CLEANUP	병커에서 열쇠를 가진 몬스터를 찾아라	KILL	50	20	-1	6		
				6	1	CH1_FIND_KEY_ITEM	열쇠 아이템을 획득한다	ITEM	20	10	-1	7		
				7	2	CH2_ENTER_E_BUILDING	열쇠로 E동에 들어간다	MOVE	10	5	-1	8		
				8	2	CH2_CLEAR_E_BUILDING	E동의 몬스터를 제거	KILL	50	30	-1	9		
				9	2	CH2_KILL_DESKMON	문서를 가진 데스크몬을 제거한다	KILL	80	30	-1	10		
				10	3	CH3_RETURN_TO_TIP_WITH_DOC	문서를 가지고 상층으로 가서 교수를 찾는다.	INTERACT	20	10	-1	11		
				11	3	CH3_CLEAR_SERVER_ROOM	상층 몬스터 제거한다.	KILL	50	40	-1	12		
				12	3	CH3_KILL_DRILL	서버를 지키는 드릴몬을 제거한다	KILL	100	40	-1	13		
				13	4	CH4_ENTER_GYM	프로그래밍 종료를 위해 체육관으로 이동한다	MOVE	15	5	-1	14		
				14	4	CH4_KILL_TINO	최종 보스 타락 티노를 제거한다	KILL	200	100	-1	15		
				100	0	TUT_START	튜토리얼 시작	INTERACT	0	0	-1	101		
				101	0	TUT_KILL_ONE_MON	몬스터 한 마리 처치하기	KILL	10	100	20	102		
				102	0	TUT_USE_ITEM	회복아이템 사용해보기	ITEM	10	10	-1	103		
				103	0	TUT_BUY_ITEM	장비 아이템 구매하기	ITEM	10	10	-1	104		
				104	0	TUT_EQUIP_ITEM	장비 아이템 착용하기	ITEM	10	10	-1	105		
				105	0	TUT_COMPLETE	튜토리얼 완료	INTERACT	30	30	-1	1		

ItemTable.csv

QuestTable.csv

→ 하드 코딩 없이 콘텐츠/밸런스를 수정할 수 있어
코드도 깔끔하고, 빌드 없이 즉시 반영 할 수 있었다

DataTable 활용 코드 예

```
void Monster::Init(EWorldChannel channelId)
```

몬스터 타입으로 정보 로드

```
{
    Character::Init(channelId);
```

```
    auto data = MonsterTable::GetInst().GetMonsterById(static_cast<uint8>(_monType));
```

```
    if (!data)
```

```
{
    LOG_W("Invaild");
    return;
}
```

```
FVector newPos{};
```

```
void Player::UseSkill(ESkillGroup groupId)
```

```
{
    auto curSkill = _info.GetSkillData(groupId);
    if (curSkill == nullptr || !curSkill->IsValid())
    {
        LOG_W("Invaild!");
        return;
    }
}
```

스킬 타입으로 정보 로드

```
uint32 level = curSkill->SkillLevel;
```

```
const FSkillTableData* skilltable = PlayerSkillTable::GetInst().GetSkill(static_cast<uint32>(groupId), level);
```

```
if (!skilltable)
```

```
{
    LOG_W("Invaild!");
    return;
}
```

```
LOG_I("Use Skill: %d", static_cast<uint8>(_monType));
```

```
bool Player::SetCurrentQuest(QuestType quest) 퀘스트 타입으로 정보 로드
```

```
{
    const QuestData* questData = QuestTable::GetInst().GetQuest(quest);
    if (!questData)
```

```
{
    LOG_W("Invalid quest ID");
    return false;
}
```

```
void Item::Init(uint8 itemTypeID)
```

아이템 타입으로 정보 로드

```
{
    itemTypeID = itemTypeID;
    _itemInfo = ItemTable::GetInst().GetItemById(itemTypeID);
    if (!_itemInfo) {
        LOG_W("Invalid ItemType");
        return;
    }
}
```

티유월드

4) 그 외 - DB 시스템

```
class DBManager
```

```
{
public:
```

```
    static DBManager& GetInst() {
        static DBManager inst;
        return inst;
    }
```

```
    bool Connect(const std::string& host, const std::string& user, const std::string& pass, const std::string& schema);
```

```
    void Shutdown();
```

```
    DBLoginResult SignUpUser(int32 sessionId, const std::string& login_id, const std::string& password, const std::wstring& nickname);
```

```
    DBLoginResult CheckLogin(int32 sessionId, const std::string& login_id, const std::string& password);
```

```
    bool UpdatePlayerInfo(uint32 dbId, const FInfoData& info);
```

```
    bool AddUserItem(uint32 dbId, uint8 itemTypeID);
```

```
    bool RemoveUserItem(uint32 dbId, uint32 itemID);
```

```
    bool DoesUserExist(uint32 userId);
```

```
    ResultCode AddFriendRequest(uint32 fromId, uint32 toId);
```

```
    bool IsFriendOrPending(uint32 userId, uint32 targetId);
```

```
    std::pair<ResultCode, std::optional<FFriendInfo>> AcceptFriendRequest(uint32 fromId, uint32 toId);
```

```
    ResultCode RejectFriendRequest(uint32 fromId, uint32 toId);
```

```
    ResultCode RemoveFriend(uint32 userId, uint32 friendId);
```

```
    std::pair<ResultCode, std::vector<FFriendInfo>> GetFriendList(uint32 userId);
```

```
    int32 FindUserDBId(const std::wstring& nickname);
```

```
};
```

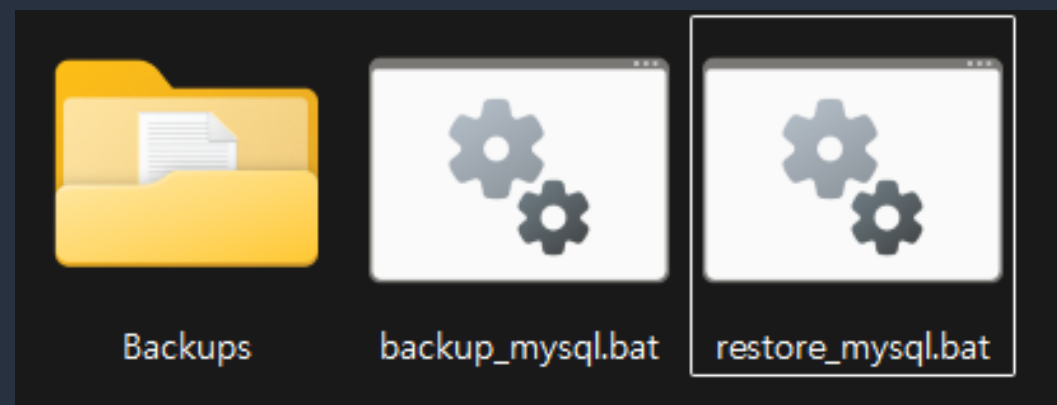
DBManager로 게임 데이터 관리

계정

인벤토리

친구

테스트 시 로그인/데이터 초기화를 매번 CMD 명령을 치기 번거로운데 자동화 할 수 없을까?



→ 배치 스크립트로 작업을 자동화해 한 번에 실행 가능하게 만들고,
클라우드 서버 환경에서 팀원들도 쉽게 사용할 수 있었다.

PROJECTS

기타프로젝트 / 다크나이트

횡스크롤 액션 슈팅 게임



개발기간

24.11 (5주)

개발도구

C++, WinAPI

개발인원

3명 (서버1, 클라이언트 2명)

담당업무

네트워크 프레임워크 제작

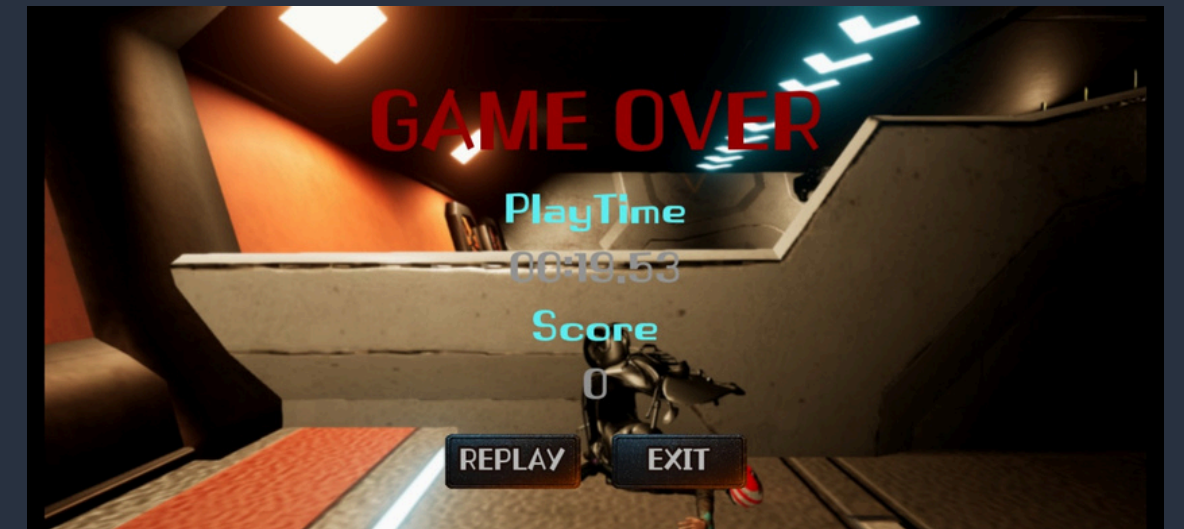
Git

<https://github.com/sujissi/NetworkGameProgramming.git>

PROJECTS

기타프로젝트 / GravityRacing

중력반전 러너 게임



개발기간		25.08 (2주)
개발도구		Unreal5.5
개발인원		3명 (클라이언트 2, 모델링 1)
담당업무		UI/사운드 처리, 앱 스토어 등록
Git		https://github.com/sujissi/GravityRacing.git

PROJECTS

기타프로젝트 / BombGame

폭탄 던지기 게임



개발기간 | 25.05 ~ 06 (2주)

개발도구 | Unreal5.5

개발인원 | 1명

개발내용 | 커스텀 기능, 채팅, 폭탄 구현 <https://youtu.be/WOgtC9ySq3c?si=sCZI8e2O7WTWxJf>

Git | <https://github.com/sujissi/BombGame.git>

감사합니다

hwangsuji329@gmail.com